CS3230 Lecture 13

Summary Lecture

(ksung@comp.nus.edu.sg)

Lecture Topics

- Revision of Amortized Analysis
- Clearer concept on NP and co-NP
- Application of Algorithm: An example in computational biology
- Advertisement of algorithm related stuffs @ SoC

if all you have is a hammer,
everything looks like a nail

HW3-S2

- We want a data-structure that supports 3 operations:
  - INSERT(S,x): inserts x into set S
  - FIND-MIN(S): returns the smallest value in set S
  - DELETE-LARGER-HALF(S): deletes the larger $\left\lceil \frac{|S|}{2} \right\rceil$ elements from S

Example

5				
5	3			
Return 3				
5	3	7		
5	3	7	2	
5	3	7	2	8
3	2			
3	2	9		
3	2	9	6	
3	2			

- Insert(S,5)
- Insert(S,3)
- Find-Min(S)
- Insert(S,7)
- Insert(S,2)
- Insert(S,8)
- Delete-Larger-Half(S)
- Insert(S,9)
- Insert(S,6)
- Delete-Larger-Half(S)

Algorithm

	1				N
A					
MV					
n					

- Data-structure
  - A table A[1..N]
  - A variable MV that stores the minimum value
  - A variable n that stores the number of values in the table
- Insert(S, x)
  - n = n + 1;
  - Set A[n] = x;
  - If (x ≤ MV), then set MV=x;
- Find-Min(S)
  - Return MV;
- Delete-Larger-Half(S)
  - v = Median-Find(A[1..n]); /* assume it finds the $\left\lceil \frac{n}{2} \right\rceil$-th largest value */
  - Delete all values ≥ v and pack the table;
  - Set n = $\left\lceil \frac{n}{2} \right\rceil$;

Worst case running time

- Insert(S, x)
  - n = n + 1;
  - Set A[n] = x;
  - If (x ≤ MV), then set MV=x;

  O(1) time

- Find-Min(S)
  - Return MV;

  O(1) time

- Delete-Larger-Half(S)
  - v = Median-Find(A[1..n]);
  - Delete all values ≥ v and pack the table;
  - Set n = $\lfloor \frac{n}{2} \rfloor$;

  Dn + 1 = O(n) time

Aggregate method

- The ith Insert costs 1 step.
- Each Find-Min costs 1 step.
- What is the cost of the jth Delete-Larger-Half?
  - Let t_j be the number of Inserts between (j-1)[th] and j[th] Delete-Larger-Half.

	t_1	t_2	t_3	t_4	t_5
Cost of 1st Delete	Dt_1				
Cost of 2nd Delete	$D\frac{t_1}{2}$	Dt_2			
Cost of 3rd Delete	$D\frac{t_1}{4}$	$D\frac{t_2}{2}$	Dt_3		
Cost of 4th Delete	$D\frac{t_1}{8}$	$D\frac{t_2}{4}$	$D\frac{t_3}{2}$	Dt_4	
Cost of 5th Delete	$D\frac{t_1}{16}$	$D\frac{t_2}{8}$	$D\frac{t_3}{4}$	$D\frac{t_4}{2}$	Dt_5
Total cost	$2Dt_1$	$2Dt_2$	$2Dt_3$	$2Dt_4$	$2Dt_5$

$= 2D\sum_i t_i$

Amortized cost

- So, we have
  - Cost of T Insert = O(T)
  - Cost of P Find-Min = O(P)
  - Cost of Q Delete-Larger-Half = O(Q + 2DT)
- The total cost is O((1+2D)T+P+Q).
- The amortized cost per operation is $O\left(\frac{(1+2D)T+P+Q}{T+P+D}\right) = O(D)$, which is constant.

Accounting method

- Insert(S, x)
  - n = n + 1;
  - Set A[n] = x;
  - If (x ≤ MV), then set MV=x;

  \$1 + \$2D

  \$2D is for median finding and table packing
  Why 2? (will explain later)

- Find-Min(S)
  - Return MV;

  \$1

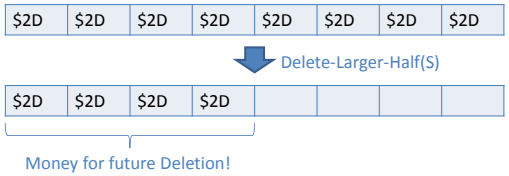
- Delete-Larger-Half(S)
  - v = Median-Find(A[1..n]);
  - Delete all values ≥ v and pack the table;
  - Set n = $\lfloor \frac{n}{2} \rfloor$;

  \$1

Note: We only allocate constant \$\$\$ to each operation.
So, the amortized cost is O(1)!

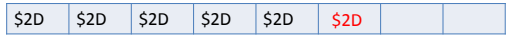
Why $2D ?

- Suppose the table A has n entries.
- To perform Delete-Larger-Half(S), it costs $Dn.
- Who contribute the money ? It is from the $\left\lceil \frac{n}{2} \right\rceil$ entries we removed. Hence, we need $2D per entry.



Analysis

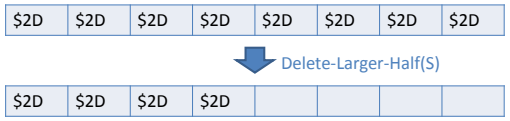
- The table is initially empty. We maintain the invariant that each non-empty entry has $2D.



- For Insert(S,x), we obtain $(1+2D). $1 is for insertion. The remaining $2D is stored in the entry.
- For Find-Min(S), we obtain $1. $1 is for finding minimum.

Analysis

- For Delete-Larger-Half(S), the table A has n entries. Each entry has $2D.
- From $\left\lceil \frac{n}{2} \right\rceil$ entries, we obtain $Dn. We use $Dn for median finding and remove half entries.
- We maintain the invariant that each non-empty entry has $2D.



Summary

- The amortized cost per operation is O(D) time.

HW3-S3

- **Shortest path problem 1**: Consider an undirected graph G=(V, E), an integer k, and two nodes s and t. Is the shortest simple path between s and t shorter than k?
- **Shortest path problem 2**: Consider an undirected graph G=(V, E), an integer k, and two nodes s and t. Is the shortest simple path between s and t longer than k?
- **Euler cycle problem**: Consider an undirected graph G=(V, E). Is there a cycle in G that covers every edge in E exactly once?
- All the above problems can be solved in polynomial time.
- All three problems are in P.
- Note that P is a subclass of NP and co-NP.
  - (Many students don't know this.)
- So, all three problems are in P, NP and co-NP.

HW3-S3

- **Hamiltonian cycle problem**: Consider an undirected graph G=(V, E). Is there a simple cycle in G that covers every vertex in V exactly once?
- This problem is in NP.
  - The certificate is a cycle C.
  - We can verify if C is a Hamiltonian cycle in linear time.
- Some student said this problem is NP-complete. Hence, it is not in P, NP and co-NP.
  - Note that all NP-complete problem are in NP.

HW3-S3

- **Longest path problem 1**: Consider an undirected graph G=(V, E), an integer k, and two nodes s and t. Is the longest simple path between s and t shorter than k?
- This problem is in co-NP.
  - Given a certificate, we can verify if the answer is "no".
  - The certificate is a simple path P between s and t.
  - We can check if P is longer than k in linear time.
- Many students said this problem is in NP.
  - It is not correct!
  - Their certificate is a path P between s and t of length < k.
  - However, this certificate cannot guarantee there is no path of length longer than k.

HW3-S3

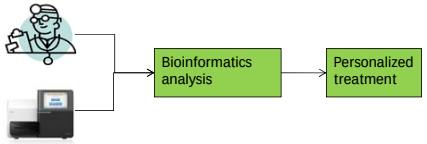
- **Longest path problem 2**: Consider an undirected graph G=(V, E), an integer k, and two nodes s and t. Is the longest simple path between s and t longer than k?
- This problem is in NP.
  - Given a certificate, we can verify if the answer is "no".
  - The certificate is a simple path P between s and t.
  - We can check if P is longer than k in linear time.
- This problem is not in co-NP.

Summary of what you learn

- First half (Analysis of Algorithm):
  - How to analyze an algorithm
    - Telescoping
    - Recursion Tree
    - Master's theorem
  - Divide and Conquer paradigm
  - O(n log n) lower bound of sorting and how to beat it
  - Median finding
  - Amortized analysis
- Second half (Design of Algorithm):
  - NP-complete
  - Graph algorithm (DFS, Strongly connected component)
  - Dynamic Programming
  - Greedy Algorithm
  - Approximation algorithm

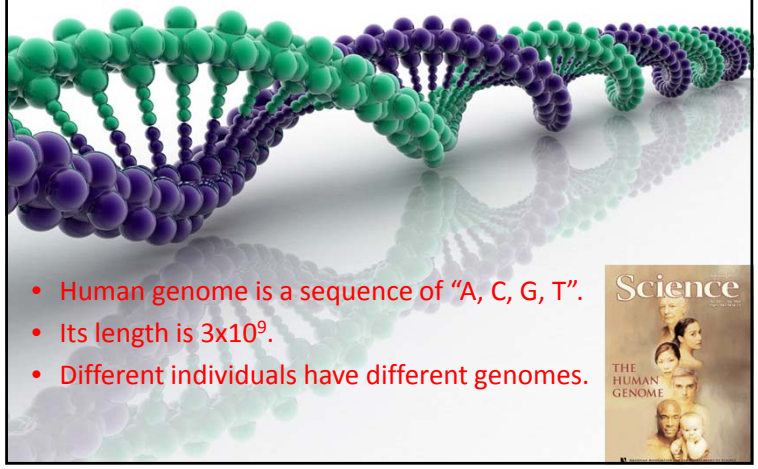
Applications of Algorithm: An example in computational biology

- Computational Biology is one of the big data problem.
- It promises to give changes to precision medicine and wellness.



- Algorithm is one of them main driving force.
- Here, we discuss one of the important problems:
  - Genome assembly

Human Genome: The Blueprint of Life



- Human genome is a sequence of "A, C, G, T".
- Its length is 3×10^9.
- Different individuals have different genomes.

Sequencer

- Genome is too long, no machine can read the whole genome.
- We have machines that can read short pieces of our genome.

Illumina Sequencers

Pacific BioSciences

Oxford Nanopore

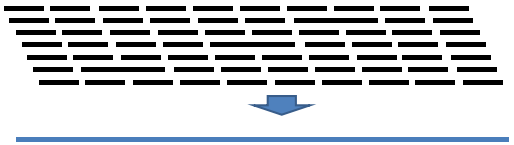
Ion Torrent

Sequencer has high throughput

- For example, one HiSeq X gives 1.8 Tera bases per day.
- Genomics data should be one of the biggest data-source nowadays!

Genome assembly problem

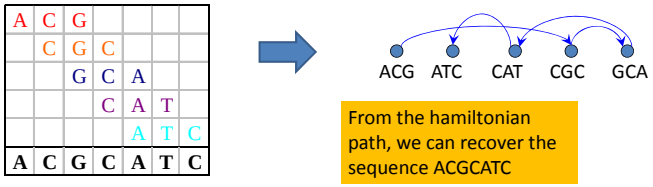
- Given the short pieces of DNA, we aim to stitch them and reconstruct the whole genome.



- This problem is important since we expect everyone will sequence his/her genome in the future.

K-mer puzzle and Hamiltonian path

- This problem can be stated as a k-mer puzzle.
- Transformation to a graph problem: Given a set of k-mers, creating a graph G with
  - each vertex representing a length-k probe in S.
  - Two vertex are connected if their corresponding length-k probes overlap by (k-1) bases.



From the hamiltonian path, we can recover the sequence ACGCATC

Trouble!

- Hamiltonian path problem is NP-complete!
- To assemble a human genome
  - Typically, there are about one billions k-mers.
  - This means that the transformed graph will have one billions of vertices.
- The current solution for Hamiltonian path problem requires $O(2^n)$ time, where $n=10^9$.

Solution

- Transform Hamiltonian path to Eulerian path.
  - Hamiltonian path problem is NP-complete.
  - Eulerian path problem can be solved in polynomial time!

Pavel A. Pevzner

Michael Waterman

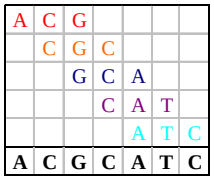
Haixu Tang

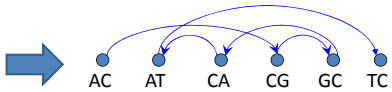
Idea

- When we transform k-mer puzzle to Hamiltonian path problem,
  - Every k-mer is represented by a vertex.
  - Then, we aim to find a Hamiltonian path that visit all k-mers.
- Instead, why don't we represent every k-mer as an edge.
- Then, our problem is to find an Eulerian path that visit all these edges.

Eulerian path solution

- Transform every k-mer as an edge.
- E.g. ACG → an edge between AC and CG.
- Transformation: Given the k-mers, creating a graph G with
  - Each vertex representing a (k-1)-prefix or (k-1)-suffix of any length-k probe in S
  - For each length-k probe, creating an edge connecting the vertices representing the (k-1)-prefix and (k-1)-suffix.





From the Eulerian path, we can recover the sequence ACGCATC

Algorithm: Thinking out of the box!

- This problem demonstrates the power of algorithm.
- Is the problem solved?
- No!
  - There are sequencing errors
  - There are repeats
  - …
- Genome assembly is still one of the most difficult and most important problem nowadays!
- We still need more interesting ideas.

Final Exam Overview

- The weightage of the final exam is 40%
- There are 5 questions, 20+15+15+15+15 = 80 marks
- The format is the same as your term test.
- It covers all topics.

Acknowledgement

- Thanks to Dr. Steven Halim.
  - Although the slides are originally written by me, it is polished by Steven.
  - Steven also provided online web tools to help to understand the material.
- Behind the scene,
  - Prof. Leong leads the teaching team
  - Nam Ninh prepares all programming assignments
  - Shunhao verifies and provides interesting ideas
- On behalf of Prof. Leong, Nam Ninh and Shunhao,
  - I would also like to thanks all of you for putting effort on this module.
  - Spending time to handle the programming tasks, assignments and tutorials.
  - We hope that you can apply what you learnt in the future.

Advertisement

If you like CS3230, you will (probably also like):

- CS3225 – Combinatorial Methods in Bioinformatics (Ken Sung)
- CS3233 – Competitive Programming (Steven Halim)
- CS4231 – Parallel and Distributed Algorithms (Yu Haifeng)
- CS4232 – Theory of Computation (Sanjay Jain)
- CS5234 – Combinatorial and Graph Algorithms (Seth Gilbert)
- CS5238 – Advance Combinatorial Methods in Bioinformatics (Ken Sung)
- CS6234 – Advanced Algorithms (Rahul Jain)
- Do research in algorithms
  (UROP, FYP, MSc by research, or PhD)
  Current trend: Big Data, Online Algorithm, …